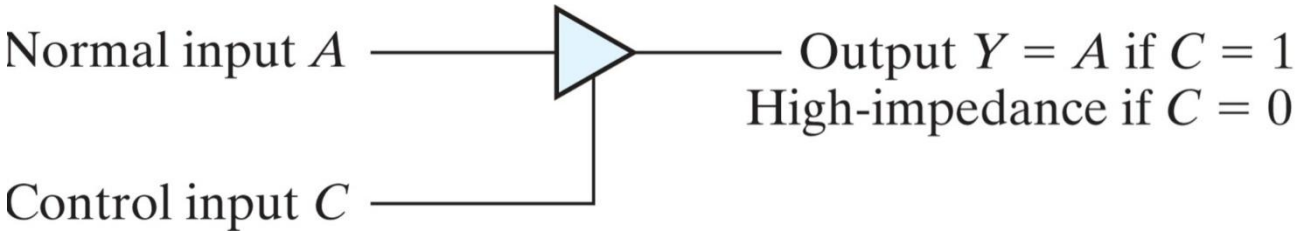
Digital Electronic Circuits (EC2L004)

Lecture 10: Combinational Logic-III & Sequential Logic

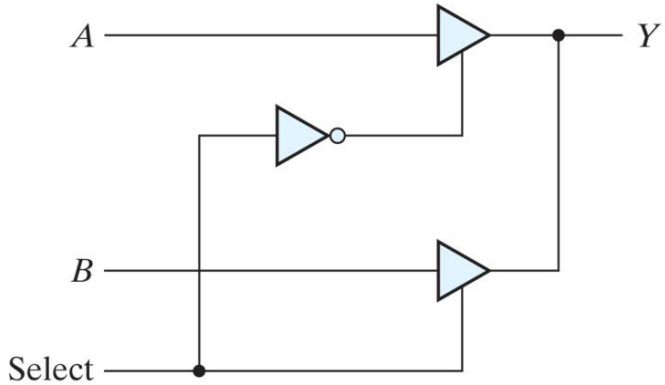
Three-State Gates

- A three-state gate is a digital circuit that exhibits three states: 0, 1 and a high-impedance (high *z* state).

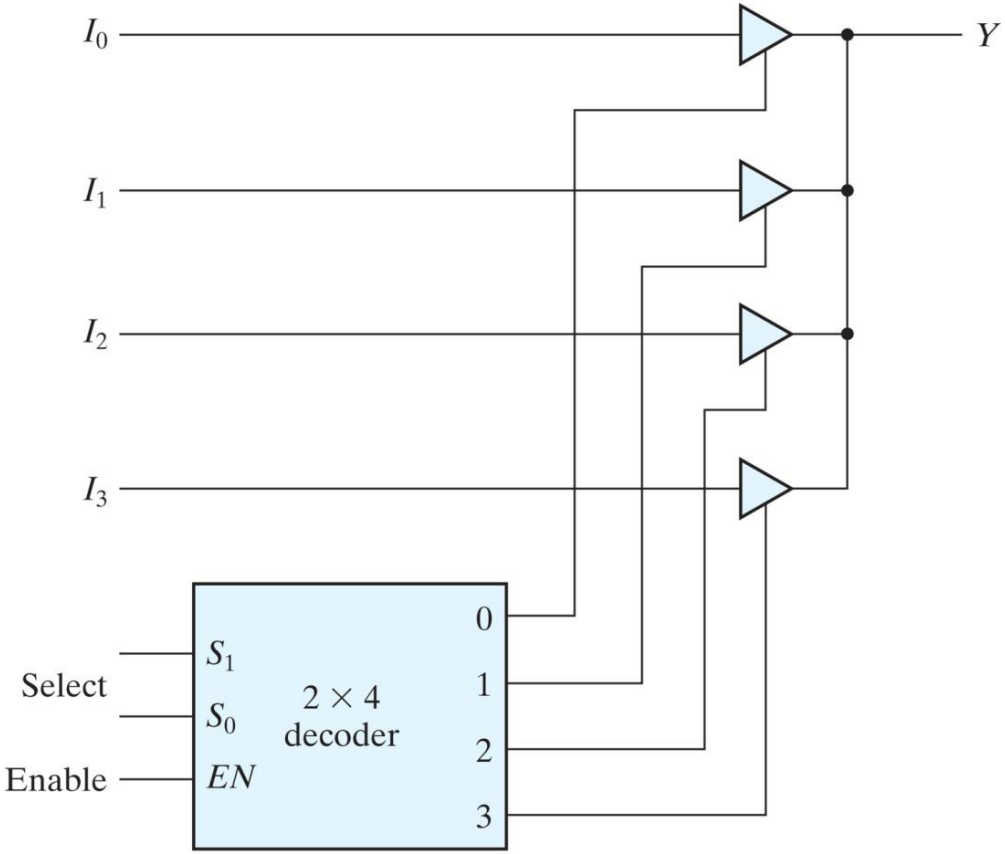
- The high impedance state the logic behaves like
  - an open circuit, which means that the output appears to be disconnected
  - the circuit has no logic significance,



Multiplexers with three-state gates



(a) 2-to-1-line mux



(b) 4-to-1-line mux

Decimal Adder

- Computers or calculators that perform arithmetic operations directly in the decimal number system represent decimal numbers in binary coded form.

- An adder for such a computer must employ arithmetic circuits that accept coded decimal numbers and present results in the same code.

- For binary addition, it is sufficient to consider a pair of significant bits together with a previous carry.

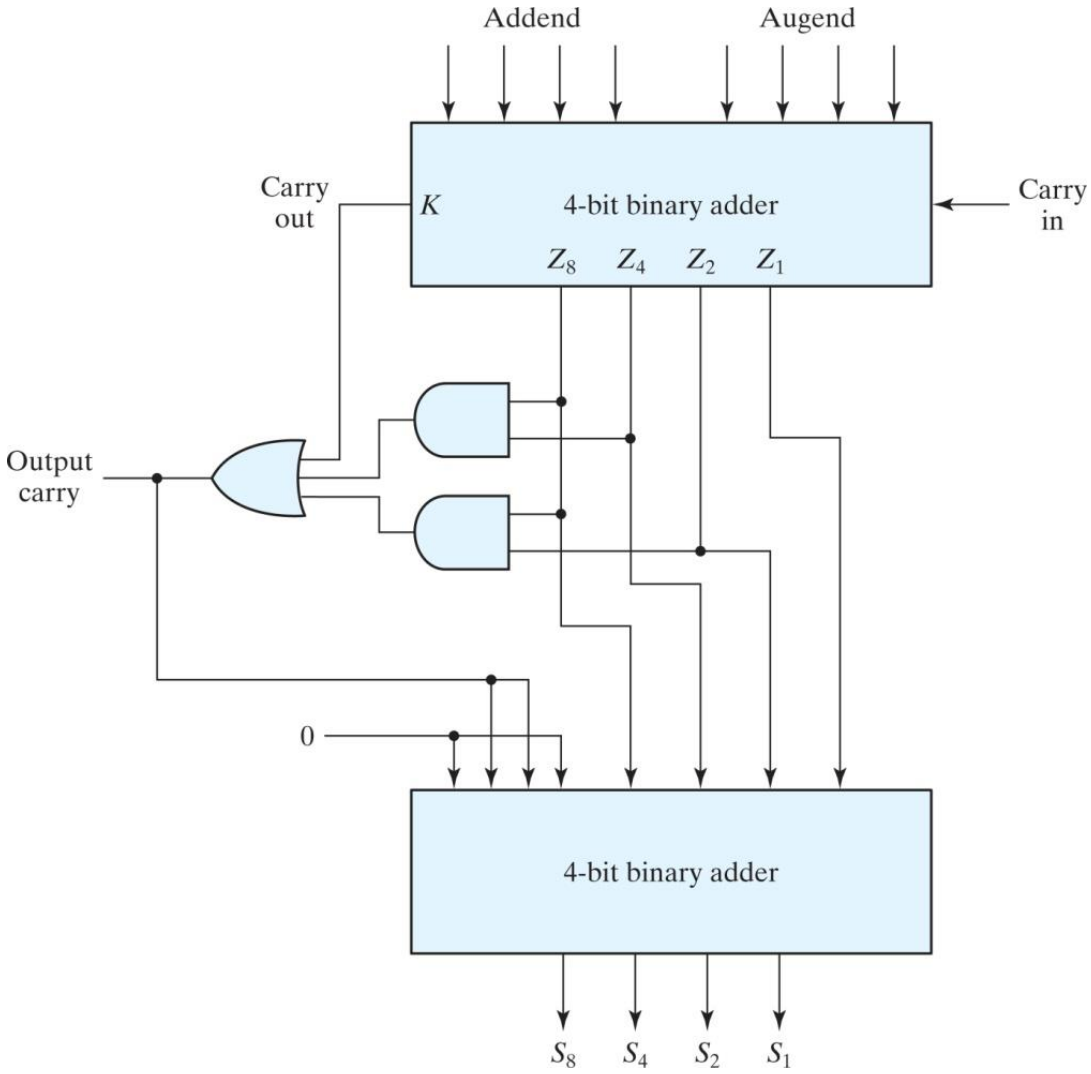
- A decimal adder requires a minimum of nine inputs and five outputs, since four bits are required to code each decimal digit and the circuit must have an input and output carry.

Decimal Adder: BCD Adder

Derivation of BCD Adder

Binary Sum					BCD Sum					Decimal
K	Z_8	Z_4	Z_2	Z_1	C	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

Decimal Adder: BCD Adder

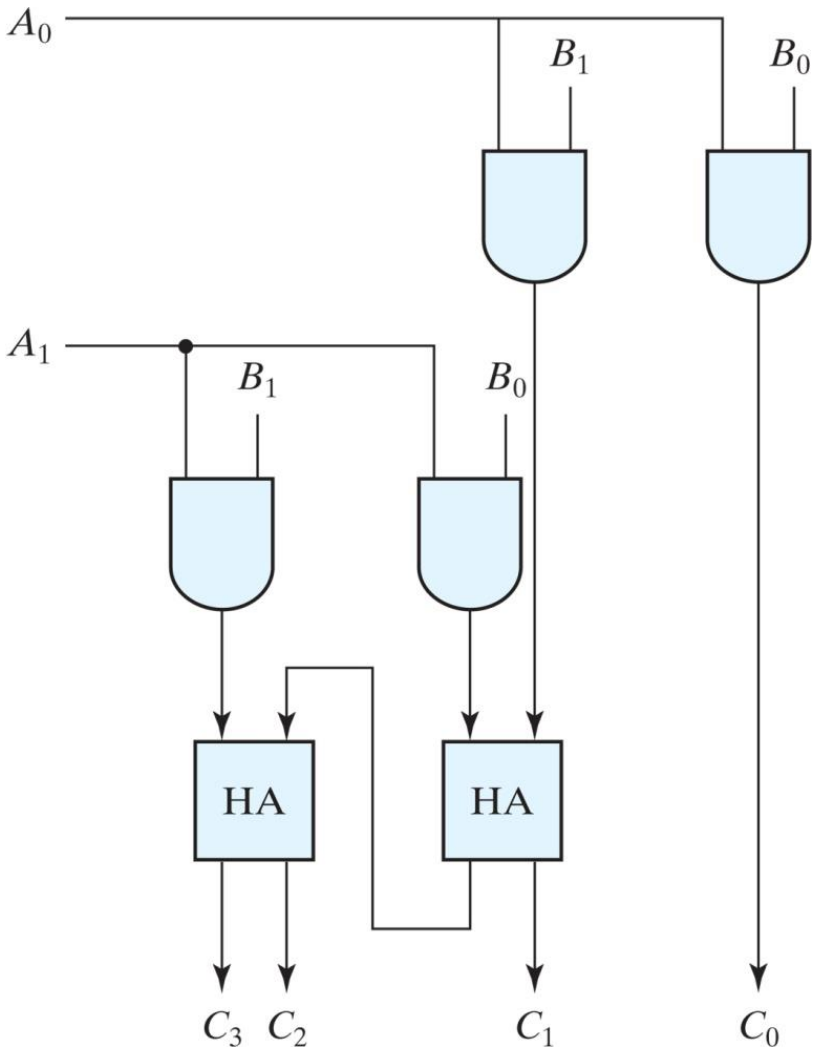


Binary Multiplier

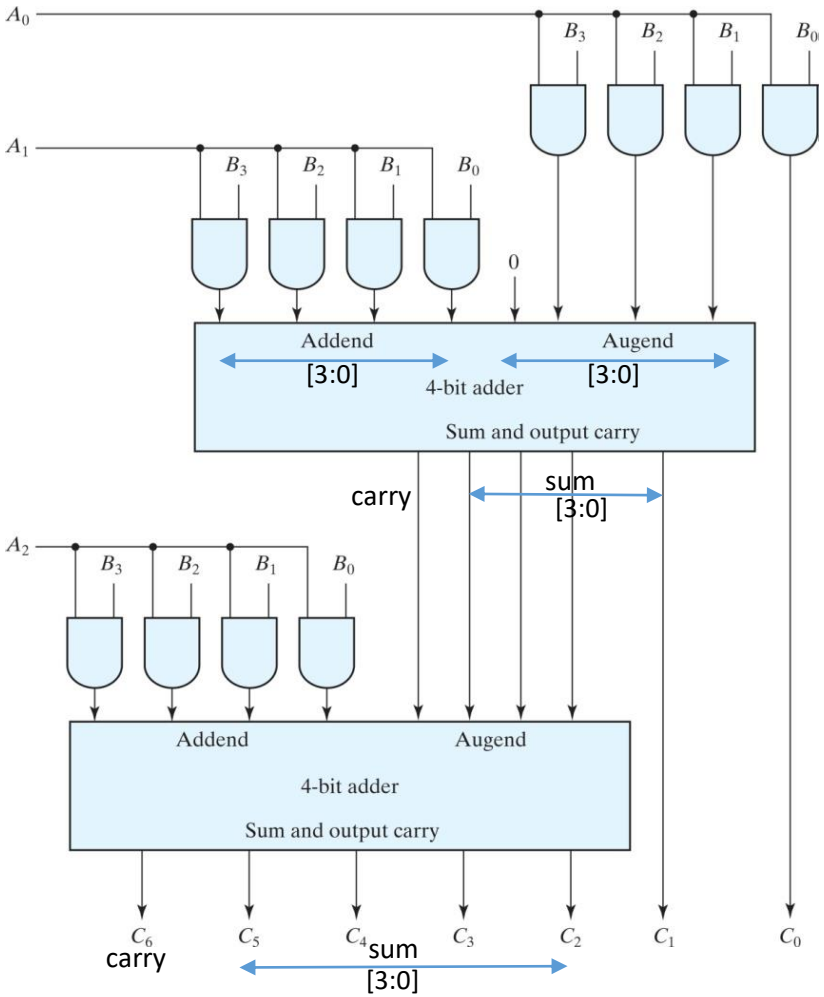
- Multiplication of binary numbers is performed in the same way as multiplication of decimal numbers.
- The multiplicand is multiplied by each bit of the multiplier, starting from the least significant bit. Each such multiplication forms a partial product.
- Successive partial products are shifted one position to the left.
- The final product is obtained from the sum of the partial products.

Two-bit by two-bit binary multiplier

		B_1	B_0
		A_1	A_0
		A_0B_1	A_0B_0
	A_1B_1	A_1B_0	
C_3	C_2	C_1	C_0



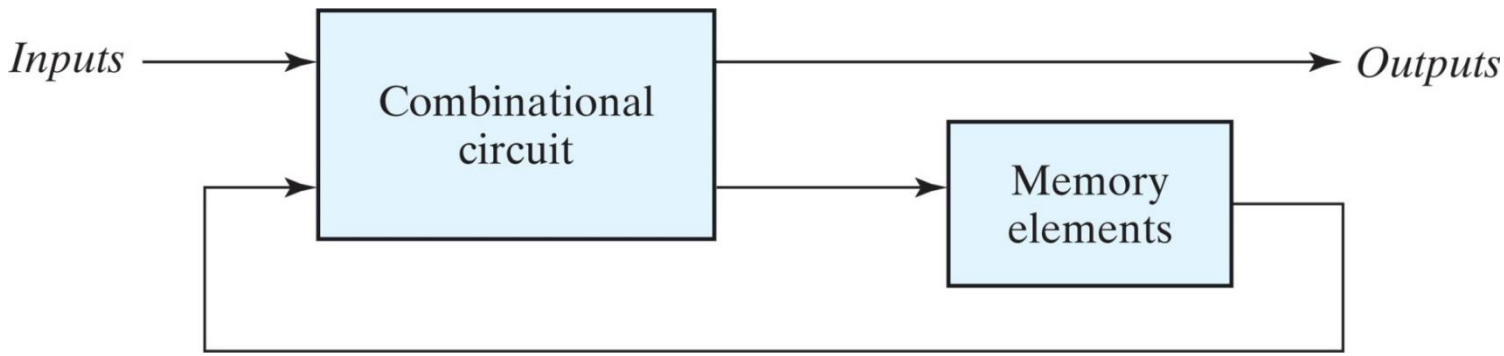
Four-bit by three-bit binary multiplier



Synchronous Sequential Logic

Sequential Circuits

- A digital system has combinational logic as well as sequential logic. The latter includes storage elements.



- The binary information stored in these elements at any given time defines the *state* of the sequential circuit at that time.

- The sequential circuit receives binary information from external inputs that, together with the present state of the storage elements, determine the binary value of the outputs.

Sequential Circuits

- These external inputs also determine the condition for changing the state in the storage elements.

- The block diagram demonstrates that
  - *the outputs in a sequential circuit are a function not only of the inputs, but also of the present state of the storage elements.*
  - *The next state of the storage elements is also a function of external inputs and the present state.*

- Thus, **a sequential circuit is specified by a time sequence of inputs, outputs, and internal states** .

- In contrast, the outputs of combinational logic depend only on the present values of the inputs.

Types of Sequential Circuits

- There are two main types of sequential circuits, and their classification is a function of the timing of their signals.

- A *synchronous* sequential circuit is a system whose behavior can be defined from the knowledge of its signals at discrete instants of time.

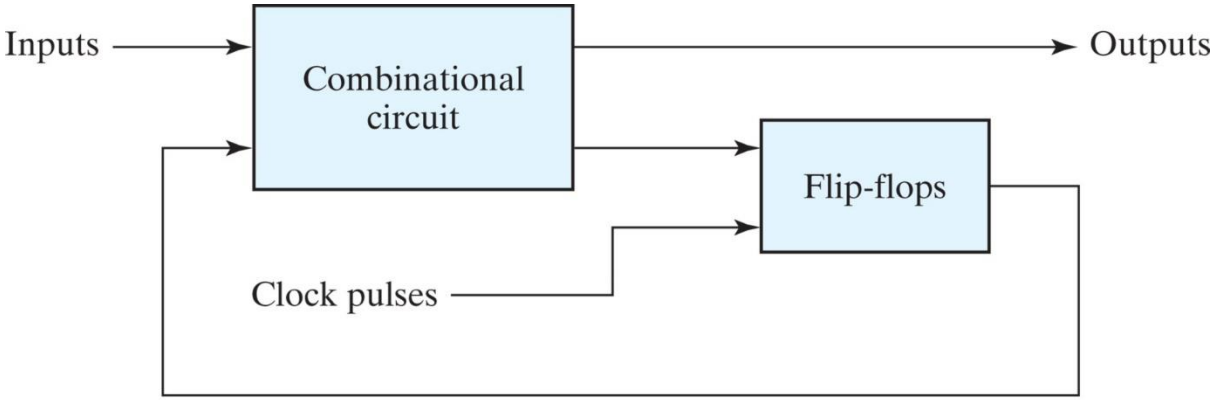
- The behavior of an *asynchronous* sequential circuit depends upon the input signals at any instant of time *and* the order in which the inputs change.

Synchronous Sequential Logic

- A synchronous sequential circuit employs signals that affect the storage elements at only discrete instants of time.
- Synchronization is achieved by a timing device called a *clock generator*, which provides a clock signal having the form of a periodic train of *clock pulses* .
- The clock signal is commonly denoted by the identifiers *clock* and *clk* .
- Synchronous sequential circuits that use clock pulses to control storage elements are called *clocked sequential circuits* and are the type most frequently encountered in practice.
  - They are called *synchronous circuits* because the activity within the circuit and the resulting updating of stored values is synchronized to the occurrence of clock pulses.

Synchronous Sequential Logic (Contd.)

- The storage elements (memory) used in clocked sequential circuits are called *flip-flops.*
- A flip-flop is a binary storage device capable of storing one bit of information.
  - In a stable state, the output of a flip-flop is either 0 or 1.



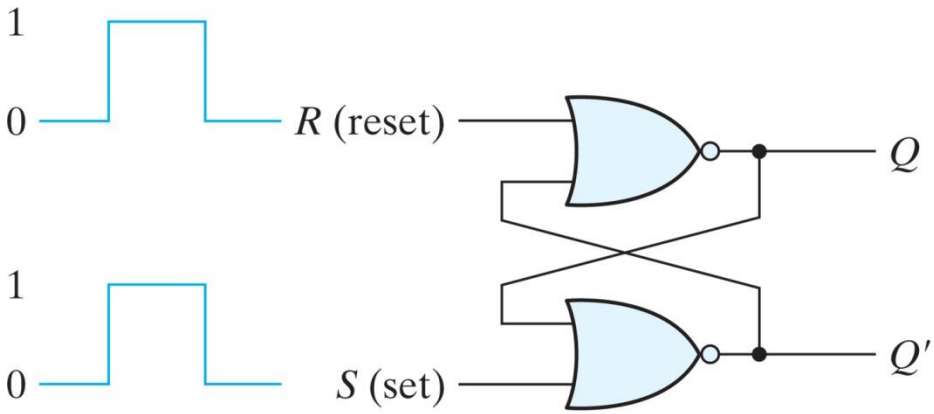
(a) Block diagram

(b) Timing diagram of clock pulses

Storage Elements: Latches

- A storage element in a digital circuit can maintain a binary state indefinitely (as long as power is delivered to the circuit), until directed by an input signal to switch states.
- The major differences among various types of storage elements are in the number of inputs they possess and in the manner in which the inputs affect the binary state.
- *Storage elements that operate with signal levels (rather than signal transitions) are referred to as latches*; *those controlled by a clock transition are flip-flops*.
- Latches are said to be level sensitive devices; flip-flops are edge-sensitive devices.
- The two types of storage elements are related because latches are the basic circuits from which all flip-flops are constructed.

SR Latch



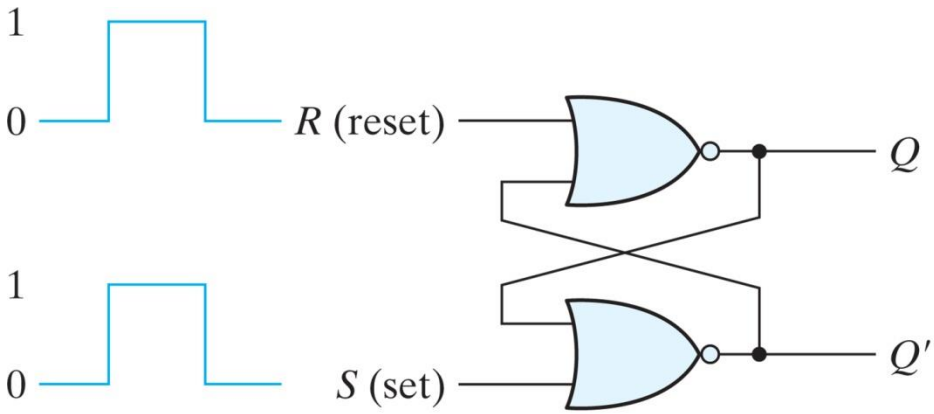
(a) Logic diagram

S	R	Q	Q'	
1	0	1	0	
0	0	1	0	(after $S = 1, R = 0$)
0	1	0	1	
0	0	0	1	(after $S = 0, R = 1$)
1	1	0	0	(forbidden)

(b) Function table

- Under normal conditions, both inputs of the latch remain at 0 unless the state has to be changed.
- The application of a momentary 1 to the *S* input causes the latch to go to the set state.
- The *S* input must go back to 0 before any other changes take place.

SR Latch



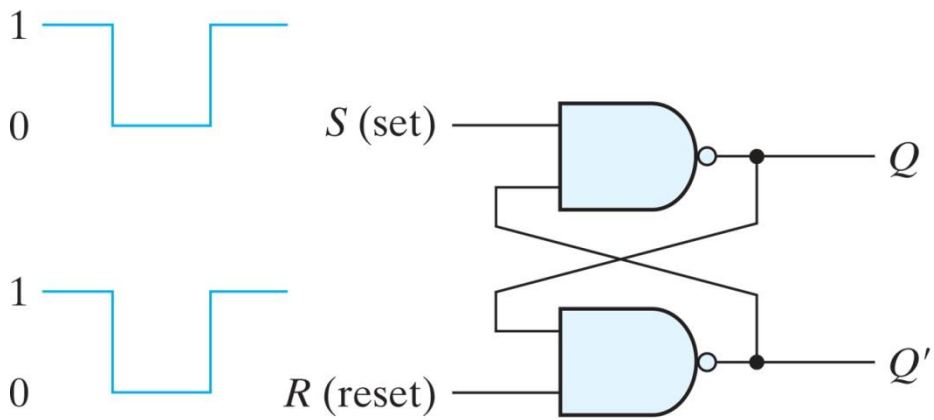
(a) Logic diagram

S	R	Q	Q'	
1	0	1	0	
0	0	1	0	(after $S = 1, R = 0$)
0	1	0	1	
0	0	0	1	(after $S = 0, R = 1$)
1	1	0	0	(forbidden)

(b) Function table

- After both inputs return to 0, it is then possible to shift to the reset state by momentary applying a 1 to the *R* input. The 1 can then be removed from *R*, where upon the circuit remains in the reset state.
- *Thus, when both inputs S and R are equal to 0, the latch can be in either the set or the reset state, depending on which input was most recently a 1.*

SR Latch / S'R' Latch



(a) Logic diagram

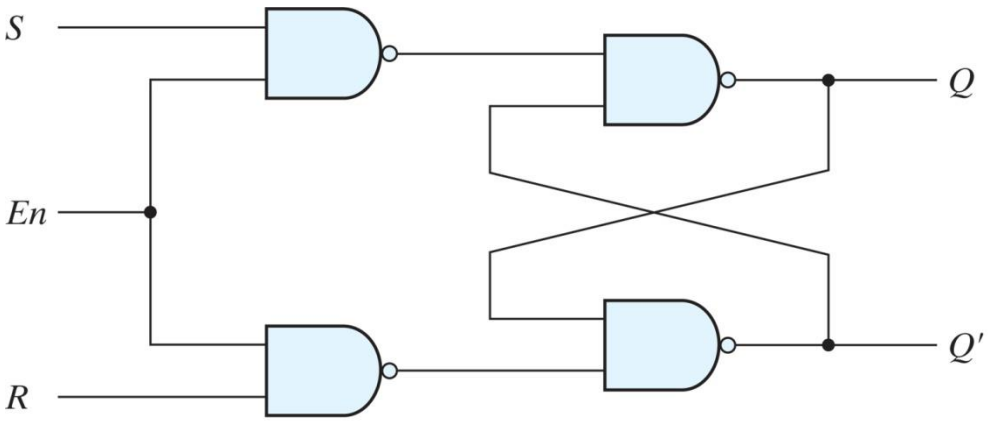
S	R	Q	Q'	
1	0	0	1	
1	1	0	1	(after $S = 1, R = 0$)
0	1	1	0	
1	1	1	0	(after $S = 0, R = 1$)
0	0	1	1	(forbidden)

(b) Function table

- SR latch with NAND gates requires a 0 signal to change its state.

- The inputs signals for the NAND-latch are the complement values used for the NOR latch.

SR Latch with control input



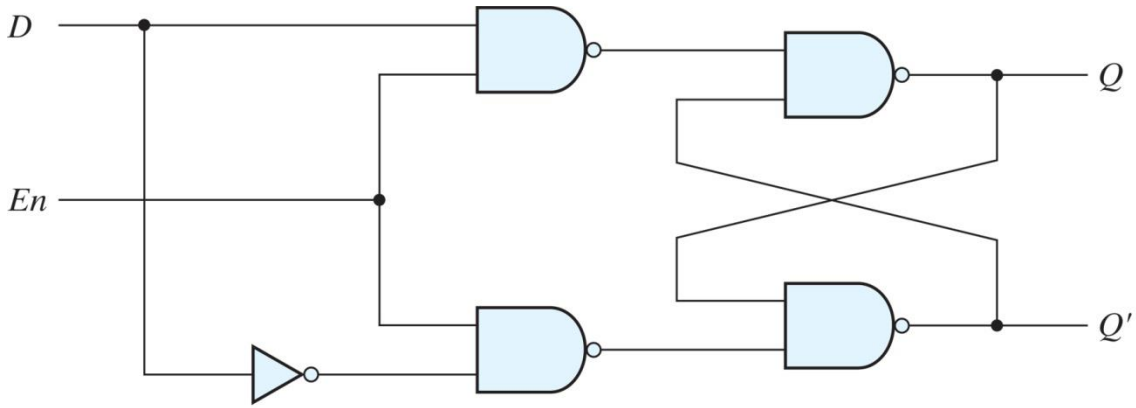
(a) Logic diagram

En	*S*	*R*	Next state of *Q*
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table

- The outputs of the NAND gates stay at the logic-1 level as long as the enable signal remains at 0.

D Latch (Transparent Latch)



(a) Logic diagram

En	*D*	Next state of *Q*
0	X	No change
1	0	$Q = 0$; reset state
1	1	$Q = 1$; set state

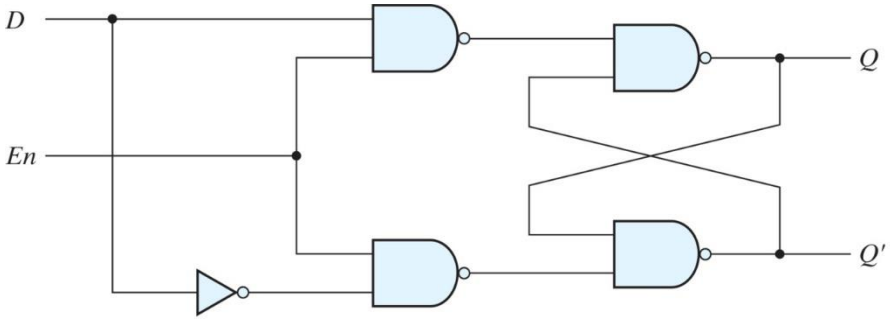
(b) Function table

- *How is D-latch structurally different than the SR latch?*
- *D* latch eliminates the undesirable condition of the indeterminate state that occurs in the *SR* latch ($Q = Q' = 1$).

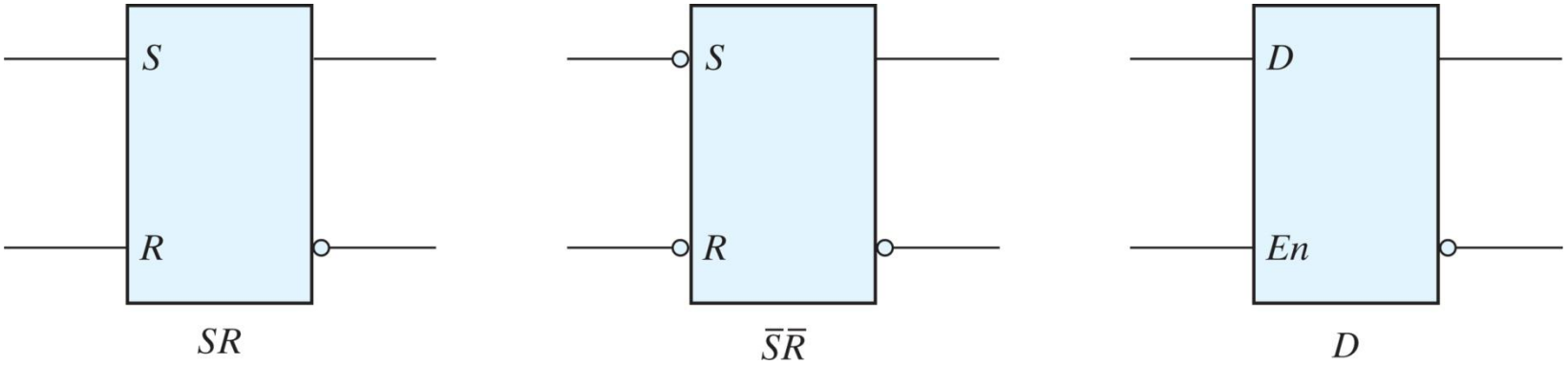
If $D = 1$, $Q = 1$ -> ‘set’ state

If $D = 0$, $Q = 0$ -> ‘reset’ state

D Latch (Contd.)



(a) Logic diagram

- The *D* latch receives that designation from its ability to hold *data* in its internal storage.
- The output follows changes in the data input as long as the enable input is asserted. This situation provides a path from input *D* to the output, and for this reason, the circuit is often called a *transparent* latch.
- When the enable input signal is de-asserted, the binary information that was present at the data input at the time the transition occurred is retained (i.e., stored) at the *Q* output until the enable input is asserted again.

Graphic Symbols for Latches